

Types for Almost-Sure Termination in the Functional Machine Calculus

Ugo Dal Lago  

University of Bologna, Italy

INRIA Sophia Antipolis, France

Willem Heijltjes  

University of Bath, United Kingdom

Abstract

We investigate almost-sure termination in higher-order probabilistic computation through the lens of the functional machine calculus. Our key observation is that iteration can easily be equipped with a non-zero chance of exiting, thus guaranteeing almost-surely terminating in a simple way. We exploit this insight by shifting attention from recursion to typed iteration, allowing probabilistic behavior to be tracked compositionally at the type level. For the purpose, we introduce a novel probabilistic type system that assigns explicit probability distributions to termination outcomes, and we show that these types soundly guarantee almost-sure termination, even in the presence of higher-order functions. We study the expressive power of the introduced system through an embedding of Markov chains, and prove that *positive* almost-sure termination does not hold at higher-order, in general.

2012 ACM Subject Classification Theory of computation → Lambda calculus; Theory of computation → Probabilistic computation

Keywords and phrases probabilistic lambda-calculus, type systems, almost-sure termination

1 Introduction

We are interested in applying the toolset of the λ -calculus, in particular confluent reduction and type systems, to probabilistic computation. In [6] we restored confluence to a probabilistic λ -calculus, the probabilistic event λ -calculus (PEL), by decomposing probabilistic choice into a *sampling* construct and a *conditional*. Independently in [1] and [15], from different inspirations we converged on essentially the same probabilistic type system for the PEL to give lower bounds on the probability of termination. In this paper we further extend the calculus, and develop a type system for *almost-sure termination* (AST): termination with probability one. In doing so, our objective differs from that of many existing works in this area, which aim to capture as many AST programs as possible [5, 4], or even to study the feasibility of capturing all such programs [16]. Instead, we seek to understand the extent to which almost-sure termination can be obtained and explained in a natural manner, possibly using well-established methods, and guaranteed by a minimal extension to simple types. More powerful methods such as dependent types and intersection types may later be used along standard lines to build on this.

We start from the following observation. *Iteration*, generally represented by while-loops, repeats a given function f until a condition is fulfilled. In the typed case, illustrated below, the function f is required to return a sum $B + A$, where the choice between returning a value A or B represents the condition. Then $\text{iter } f$ evaluates f , exits on B , and repeats on A .

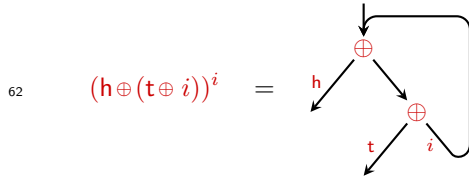
$$\frac{f : A \rightarrow B + A}{\text{iter } f : A \rightarrow B}$$

With the above rule, as with typed fixed-point combinators, typing no longer guarantees termination. However, we observe that if we replace $B + A$ by a probabilistic sum $B \oplus_p A$ with a fixed probability $p \in (0, 1]$, i.e. the probability of choosing B is non-zero and does not

depend on the input, then iter f is *almost-surely terminating* (provided of course that f is itself terminating). This opens a way to approach almost-sure termination via type systems. A crucial question arises: how can we handle a type constructor such as $B \oplus_p A$ within a λ -calculus with probabilistic choice in a compositional and elegant manner?

These considerations illustrate how probabilistic choice is connected to other imperative features such as sequentiality and iteration. Derived from the PEL, the second author has developed a new approach to unifying the λ -calculus with imperative computation and effects, the *Functional Machine Calculus* (FMC) [12]. Its most recent instance [14] features branching sequential computation and iteration, providing an ideal vehicle for our present study. In the FMC each branch of the computation is labelled with a *choice* label $i, j, k, \dots$, which takes the rôle of a constant, an exception, a case label, or an exit status. For example, the boolean constants $\perp$ and $\top$ are choice labels. Iteration of a term M is written M^i , which evaluates M and repeats if M chooses i , terminating for any other choice. The calculus encodes a fair probabilistic choice, which we write as $M \oplus N$.

► **Example 1.** The term $(h \oplus (t \oplus i))^i$ chooses *heads*, the choice label h , with probability $\frac{1}{2}$, *tails* t with probability $\frac{1}{4}$, and iterates with probability $\frac{1}{4}$, to give an overall probability of $\frac{2}{3}$ for h and $\frac{1}{3}$ for t . We illustrate it pictorially as follows; it will remain a running example throughout the paper.



The type system we introduce in this paper, simplified for this introduction to omit inputs and return values, records the probability of terminating with each given choice. A term M is typed as follows, indicating that it may choose i_k with probability p_k from the choices i_1 through i_n . For simplicity, we restrict ourselves to rational probabilities, $p, q \in [0, 1] \cap \mathbb{Q}$.

67 $M : p_1 i_1 \oplus \dots \oplus p_n i_n$

The type is a probability (sub-)distribution over the choices $\{i_1, \dots, i_n\}$, written as a formal sum. For greater generality, and since our formalism readily incorporates it, we allow the probabilities p_1 through p_n to sum to less than one, with the remainder representing an upper bound to the probability of divergence. We note here that, as a λ -calculus, the full calculus will have two means of non-termination: iteration with probability one, and higher-order non-terminating terms such as $\Omega = (\lambda x.xx)(\lambda x.xx)$. The type system will address both, and a term is almost-surely terminating if the sum probability of its type is exactly one.

► **Example 2.** The body of our example term, without the loop, has the following type.

76 $h \oplus (t \oplus i) : \frac{1}{2}h \oplus \frac{1}{4}t \oplus \frac{1}{4}i$

Iteration M^i removes the choice i from the possible return choices. In the probabilistic setting, the remaining probabilities must then be renormalized to form a distribution. That is, if i has probability q , the remaining choices (including divergence) have a total probability of $1 - q$, and must be multiplied by $\frac{1}{1-q}$ to return the total to one. Typing for iteration is

81 then as follows.

$$\begin{array}{l}
 82 \quad \frac{M : p_1 i_1 \oplus \dots \oplus p_n i_n \oplus q j}{M^j : \frac{p_1}{1-q} i_1 \oplus \dots \oplus \frac{p_n}{1-q} i_n} \quad M = \begin{array}{c} \downarrow \\ \boxed{M} \\ \swarrow \quad \downarrow \quad \searrow \\ i_1 \quad \dots \quad i_n \quad j \\ p_1 \quad \dots \quad p_n \quad q \end{array} \quad M^j = \begin{array}{c} \downarrow \\ \boxed{M} \\ \swarrow \quad \downarrow \quad \searrow \\ i_1 \quad \dots \quad i_n \quad j \\ \frac{p_1}{1-q} \quad \dots \quad \frac{p_n}{1-q} \quad \frac{p_n}{1-q} \end{array}
 \end{array}$$

83 ► **Example 3.** The type for our example term is given by taking the type $\frac{1}{2}h \oplus \frac{1}{4}t \oplus \frac{1}{4}i$
 84 of the body of the loop, removing the element $\frac{1}{4}i$, and then multiplying the remaining
 85 sub-distribution $\frac{1}{2}h \oplus \frac{1}{4}t$ by $\frac{4}{3}$ to renormalize it, which yields the following.

$$86 \quad (h \oplus (t \oplus i))^i : \frac{2}{3}h \oplus \frac{1}{3}t$$

87 Types for iteration then correctly give the probabilities for each choice. Consider M^j
 88 where j has probability q in M . For any other choice i with probability p in M the chance of
 89 M^j choosing i is given by the following geometric series, generated by choosing i immediately,
 90 after one iteration, after two iterations, and so forth. The limit of this series is the probability
 91 given by the type system.

$$92 \quad p + pq + pq^2 + pq^3 + \dots = \frac{p}{1-q}$$

93 In this paper, we explore this idea in the context of the FMC, introducing the *probabilistic FMC* (PFMC). This combines several key computational notions in a single cohesive
 94 package: higher-order functions through the λ -calculus, which conservatively embeds as a
 95 fragment; branching and iteration via choice labels, as introduced in the FMC [14]; and
 96 the decomposition of probabilistic choice of the probabilistic event λ -calculus [6]. The main
 97 contribution is the typed calculus, the PFMC $^{\Rightarrow\oplus}$, a probabilistic variant of the typed FMC,
 98 with the main result that it guarantees almost-sure termination.

100 An overview of the features of the calculus, in order of appearance in the paper, is
 101 as follows. We introduce the untyped PFMC as a fragment of the FMC in Section 3
 102 via its operational semantics, a simple stack machine that naturally extends the Krivine
 103 machine [17]. The calculus inherits confluent reduction from the FMC [14]; while a notable
 104 feature, we omit it here as it plays no crucial rôle. The typed calculus PFMC $^{\Rightarrow\oplus}$, introduced
 105 in Section 4, conservatively extends simple types for the λ -calculus (Remark 16) and our
 106 previous probabilistic types for the PEL [1, 15] (Remark 17). The main result, Theorem 21,
 107 is that the probability given by the type of a term is a lower bound on the probability of
 108 termination, which implies almost-sure termination if that probability is one.

109 In the remainder of the paper we then consider two aspects of the expressive power of
 110 the calculus. Section 5 considers *ground terms*, the fragment of the calculus that combines
 111 branching and iteration with probabilistic choice, omitting the λ -calculus; essentially the
 112 fragment we have been using in this introduction. We observe that ground terms are
 113 equivalent to finite Markov chains (Proposition 26), and that typing, which is decidable,
 114 computes the return distribution of an encoded Markov chain (Proposition 28). This fragment
 115 is *positive* AST (PAST): the expected runtime for a probabilistic term on the machine is
 116 finite. The full PFMC $^{\Rightarrow\oplus}$, being higher-order, is more expressive, and in Section 6 we give an
 117 example term that is *null* AST (NAST): while the probability of termination is one, expected
 118 runtime is infinite. The mechanism is standard: by probabilistically generating a series of
 119 terms M_n as below, with exponentially decreasing probability but exponentially increasing
 120 runtime, i.e. M_{n+1} takes double the time of M_n .

$$121 \quad \frac{1}{2}M_1, \frac{1}{4}M_2, \frac{1}{8}M_3, \dots$$

122 2 Related Work

123 The type system introduced here can be seen as a conservative extension of the simple type
 124 systems studied in [1] and [15]. While those systems already featured higher-order functions,
 125 they did not account for either iteration or recursion. In the present work, we deliberately
 126 choose *iteration* rather than *recursion* in order to keep the framework as simple as possible.
 127 Indeed, handling probabilistic recursion poses substantial technical challenges, as it typically
 128 requires probabilistic function variables at the type level, effectively leading to the use of
 129 dependent types.

130 Several other works [2, 5, 7, 8] introduce type systems that guarantee almost-sure
 131 termination (AST) or positive almost-sure termination (PAST) for probabilistic λ -calculi
 132 equipped with recursion. However, these approaches generally prioritise expressive power,
 133 often at the expense of conceptual and technical simplicity. A notable exception is the work
 134 of the first author with Breuvart and Herrou on AST for extensions of Gödel's $\mathbb{T}$ [3]. In
 135 that setting, however, one endows the underlying calculus either with recursion or with
 136 sampling from infinite distributions, rather than with iteration. In contrast, iteration is a
 137 fundamentally new ingredient of our approach and plays a central rôle in preserving simplicity
 138 while still capturing nontrivial probabilistic behaviour.

139 Another related line of research concerns the use of intersection types to characterise
 140 AST and PAST properties [4]. Although such characterisations are indeed possible, they
 141 come at a significant cost: not only is type inference undecidable, but the resulting type
 142 structures are also highly complex. Yet another body of work focuses on the underlying
 143 verification problem. Unlike the deterministic case, probabilistic termination is undecidable
 144 even for higher-order programs operating over finite datatypes [16], e.g., in probabilistic
 145 variants of the λY -calculus and higher-order recursion schemes. Our emphasis differs from
 146 these approaches, as we deliberately trade expressive power for a simpler and more tractable
 147 framework.

148 Finally, we acknowledge that the calculus introduced here currently has limited expressive
 149 power for practical algorithm design. This limitation stems both from the restriction to simple
 150 types and from the nature of exit probabilities in the iteration typing rule, which are fixed.
 151 A natural direction for future work would be a dependently typed generalisation, reminiscent
 152 of Hoare logic. Such an extension could facilitate reasoning about probabilistic termination
 153 in interactive theorem proving environments. From this perspective, the extensive body of
 154 work on sound verification of almost-sure termination in imperative programs [11, 19] and
 155 probabilistic automata [10, 9] also deserves mention. Although motivated by expressiveness
 156 and thus fundamentally different in spirit from the present work, this line of research offers
 157 many valuable ideas that could potentially be incorporated into the introduced framework.

158 3 The Probabilistic FMC

159 The Functional Machine Calculus (FMC) [12, 14] combines the λ -calculus with imperative
 160 programming and effects in the following way. Its point of departure is the (simplified)
 161 Krivine Abstract Machine [17], a call-by-name operational semantics for the λ -calculus that
 162 evaluates terms in the context of an argument stack, where *application* $M N$ pushes N , and
 163 *abstraction* $\lambda x.M$ pops the head of the stack and substitutes it for x in M . To emphasise the
 164 operational intuition, *push* is written $[N].M$ and *pop* $\langle x \rangle.M$. The calculus and the machine
 165 are then extended as follows.

166 **Effects** The machine is generalised from one to multiple argument stacks or streams, which

are then used to model effects: higher-order store as stacks of depth one, input/output as pop-only and push-only streams, and probabilistic choice as a pop-only stream of boolean values. These constructions derive from the probabilistic event λ -calculus (PEL) [6], the precursor to the FMC, which decomposes probabilistic choice $M \oplus N$ as $\boxed{a}.MaN$, with a generator $\boxed{a}.M$ that pops from a probabilistic stream of booleans and a conditional MaN that chooses M or N depending on a .

Sequencing The calculus is extended with imperative *skip* and *sequence* $M;N$, implemented in standard fashion on the machine with a continuation stack where $M;N$ pushes N and *skip* pops and executes. This further enables the embedding of Plotkin's call-by-value λ -calculus, the monadic *return* and *let*-binding of Moggi's computational metalanguage [20], and Levy's call-by-push-value [18].

Control The calculus is generalised from sequential to branching computation by parameterizing *skip* in a set of *choice* labels $i, j, k, \dots$, each indicating a distinct branch of the computation. *Sequence* becomes conditional on a choice $M; i \rightarrow N$, composing M with N on the branch labelled i in M . Operationally, the continuation stack becomes a stack of cases $i \rightarrow N$, where $M; i \rightarrow N$ pushes and i pops and selects or discards. This is used to model control flow constructions: conditionals, constants, case switches, and exception handling. Iteration is introduced by the construct M^i which evaluates M and iterates on the choice i , exiting otherwise, with the semantics $M^i = M; i \rightarrow M^i$.

In this way the FMC embeds a minimal but complete imperative programming language, and extends to it two primary features of the λ -calculus: confluent reduction and simple types. In this paper we consider the *probabilistic FMC* (PFMC), a fragment of the calculus with *control* and two argument stacks, the regular one for the λ -calculus and a probabilistic input stream.

► **Definition 4.** The terms of the PFMC are given by the following grammar.

$$M, N ::= x \mid \langle x \rangle. M \mid [N]. M \mid i \mid M; i \rightarrow N \mid M^i \mid \boxed{a}. M$$

The constructs are: a variable x ; a pop $\langle x \rangle. M$ that binds x in M ; a push $[N]. M$; a choice i ; a case $M; i \rightarrow N$; a loop M^i ; and a generator $\boxed{a}. M$ which binds the variable a in M .

The λ -calculus embeds by letting $\lambda x. M = \langle x \rangle. M$ and $M N = [N]. M$. The generator $\boxed{a}. M$ is rendered as $\text{rnd}\langle a \rangle. M$ in other presentations of the FMC, as a parameterised pop construct accessing a stream rnd that represents a random generator. The calculus is thus an orthogonal combination of the λ -calculus, in the fragment of *variable*, *pop*, and *push*; the *control* extension of the FMC, consisting of *choice*, *case*, and *loop*; and the generator $\boxed{a}. M$ of the probabilistic event λ -calculus.

Probabilistic choice $M \oplus N$ encodes as follows, where the *case* construct associates left, $M; i \rightarrow N; j \rightarrow P = (M; i \rightarrow N); j \rightarrow P$.

$$M \oplus N = \boxed{a}. a; \top \rightarrow M; \perp \rightarrow N$$

In this construction, first $\boxed{a}$ pops a boolean value $\top$ or $\perp$ from the probabilistic stream and substitutes it for a . The remainder then selects the relevant branch, following the semantics of the *case* construct: $i; i \rightarrow M = M$ and $j; i \rightarrow M = j$ for $i \neq j$. We reserve the choice labels $\top$ and $\perp$ for the purpose of encoding probabilistic choice, since if M were allowed to choose $\perp$ here, that would incorrectly trigger N after evaluating M .

► **Example 5.** Our running example term unfolds into the full calculus as follows.

$$(\mathbf{h} \oplus (\mathbf{t} \oplus \mathbf{i}))^i = (\boxed{a}. a; \top \rightarrow \mathbf{h}; \perp \rightarrow (\boxed{b}. b; \top \rightarrow \mathbf{t}; \perp \rightarrow \mathbf{i}))^i$$

The generator $\boxed{a}.M$ represents a *fair* coin toss, with equal probability for $\top$ and $\perp$. We may write $\boxed{a}_p.M$ for an unfair coin with probability p for $\top$ and $1-p$ for $\perp$. This construct may either be considered a primitive, generalising $\boxed{a}.M$, or constructed from fair coin tosses with iteration, in the way our running example term generates probability $\frac{1}{3}$.

The Operational Semantics

We will give the small-step and big-step operational semantics of the PFMC, the former as an abstract machine and the latter as an inductive evaluation relation. The FMC further features a confluent reduction relation [14], which we omit here.

The abstract machine is given by the Krivine machine with the extensions described at the start of this section. In addition to the standard argument stack, it will have a *trace*, a stack of boolean values that represents the probabilistic generator, and a *continuation stack* that implements the branching and iteration of the *case* and *loop* constructs. A term M is evaluated in the context of these three stacks until the machine halts with a choice term i and an empty continuation stack, returning the argument stack as output and the choice i as the exit status, indicating the chosen branch of the computation. The machine itself is deterministic; the probabilistic aspect of computation is modelled by taking the trace to be probabilistically generated.

► **Definition 6.** *The functional abstract machine is given by the following data. Traces r, s, t , argument stacks R, S, T , and continuation stacks K, L are as follows.*

$$r, s, t ::= \varepsilon \mid \perp t \mid \top t \quad R, S, T ::= \varepsilon \mid S M \quad K, L ::= \varepsilon \mid (i \rightarrow M) K$$

The states of the machine are four-tuples (t, S, M, K) . Its steps or transitions are given in Figure 1. A run of the machine is a sequence of steps, written with a double line as below. A successful run is one terminating in a final state $(\varepsilon, T, i, \varepsilon)$ with a choice term i and empty trace and continuation stack.

$$\frac{(s, S, M, K)}{(t, T, N, L)}$$

Example 30 in the Appendix demonstrates a successful run of the machine for our running example term.

The big-step operational semantics, in the form of an inductively defined evaluation relation $(\Downarrow)$, describes the successful runs of the machine directly. Where the machine implements branching computation via the continuation stack, the big-step semantics gives a more abstract view as conditional sequential composition.

► **Definition 7.** *The evaluation relation has statements $t, S, M \Downarrow T, i$ and is defined inductively by the following rules.*

$$\begin{array}{c} \frac{}{\varepsilon, S, i \Downarrow S, i} \text{ax} \quad \frac{r, R, M \Downarrow S, i \quad s, S, N \Downarrow T, j}{rs, R, M; i \rightarrow N \Downarrow T, j} \text{sel} \quad \frac{r, R, M \Downarrow S, j}{r, R, M; i \rightarrow N \Downarrow S, j} \text{rej} \\[10pt] \frac{s, S, N, M \Downarrow T, i}{s, S, [N]. M \Downarrow T, i} \text{push} \quad \frac{r, R, M \Downarrow S, i \quad s, S, M^i \Downarrow T, j}{rs, R, M^i \Downarrow T, j} \text{loop} \quad \frac{r, R, M \Downarrow S, j}{r, R, M^i \Downarrow S, j} \text{exit} \\[10pt] \frac{t, S, \{N/x\}M \Downarrow T, i}{t, S, N, \langle x \rangle. M \Downarrow T, i} \text{pop} \quad \frac{t, S, \{\perp/a\}M \Downarrow T, i}{\perp t, S, \boxed{a}. M \Downarrow T, i} \text{simpl} \quad \frac{t, S, \{\top/a\}M \Downarrow T, i}{\top t, S, \boxed{a}. M \Downarrow T, i} \text{simpl} \end{array}$$

push	$\frac{(t, S, [N].M, K)}{(t, S N, M, K)}$	$\frac{(t, S, i, (i \rightarrow M) K)}{(t, S, M, K)}$	select
pop	$\frac{(t, S N, \langle x \rangle.M, K)}{(t, S, \{N/x\}M, K)}$	$\frac{(t, S, i, (j \rightarrow M) K)}{(t, S, i, K)}$	reject
sequence	$\frac{(t, S, M; i \rightarrow N, K)}{(t, S, M, (i \rightarrow N) K)}$	$\frac{(t, S, M^i, K)}{(t, S, M, (i \rightarrow M^i) K)}$	iterate
sample	$\frac{(\top t, S, [a].M, K)}{(t, S, \{\top/a\}M, K)}$	$\frac{(\perp t, S, [a].M, K)}{(t, S, \{\perp/a\}M, K)}$	

■ **Figure 1** The Abstract Machine

245 The machine and the big-step evaluation relation can be put in correspondence as follows.

246 ► **Proposition 8** ([14, Proposition 5.4]). *Evaluation characterizes successful machine termination:*

$$248 \quad t, S, M \Downarrow T, i \iff \frac{(t, S, M, \varepsilon)}{(\varepsilon, T, i, \varepsilon)}$$

249 Probabilistic Evaluation

250 The FMC as defined above is still deterministic: it externalises the probabilistic component
251 of evaluation to the input trace t and how this is generated. Here, we will define *probabilistic*
252 evaluation by considering distributions of machine runs over distributions of traces t .

253 A *frontier* F is a (possibly infinite) set of traces such that no element in F is a prefix of
254 another. The *probability* $\text{pr}(F)$ of a frontier F is the sum probability of its traces.

$$255 \quad \begin{aligned} \text{pr}(t) &= \left(\frac{1}{2}\right)^{|t|} \quad \text{where } |t| \text{ is the length of } t \\ \text{pr}(F) &= \sum_{t \in F} \text{pr}(t) \end{aligned}$$

256 ► **Example 9.** The set of traces $\{(\perp^n)\top \mid n \in \mathbb{N}\} = \{\top, \perp\top, \perp\perp\top, \dots\}$ is a frontier with
257 probability 1.

258 A *return family* $\mathcal{S}$ gives the result of successfully evaluating a term for every trace in a
259 given frontier F , collecting for each $t \in F$ a return stack S_t and choice i_t as the pair $(S_t, i_t)_t$
260 indexed with t . The frontier F over which $\mathcal{S}$ is defined is written $[\mathcal{S}] = F$, and the *probability*
261 of a return family is that of its frontier, $\text{pr}(\mathcal{S}) = \text{pr}([\mathcal{S}])$.

$$262 \quad \mathcal{S}, \mathcal{T} ::= \{(S_t, i_t)_t\}_{t \in F}$$

263 ► **Definition 10.** A probabilistic evaluation $S, M \Downarrow \mathcal{T}$ of a term M and stack S to a return
264 family $\mathcal{T}$ is given by the evaluation of M and S with each $t \in F$, as follows.

$$265 \quad S, M \Downarrow \{(T_t, i_t)_t\}_{t \in F} \iff \forall t \in F. (t, S, M \Downarrow T_t, i_t)$$

266 ► **Example 11.** For our running example term we have the evaluation $\varepsilon, (\mathbf{h} \oplus (\mathbf{t} \oplus i))^i \Downarrow \mathcal{T}$ for
267 the following return family $\mathcal{T}$ over the frontier of Example 9.

$$268 \quad \mathcal{T} = \{(\varepsilon, \mathbf{h})_t \mid t = (\perp^{2n})\top, n \in \mathbb{N}\} \cup \{(\varepsilon, \mathbf{t})_t \mid t = (\perp^{2n+1})\top, n \in \mathbb{N}\}$$

Observe that evaluation is closed under inclusion: if $R, M \Downarrow \mathcal{T}$ and $\mathcal{S} \subseteq \mathcal{T}$ then $R, M \Downarrow \mathcal{S}$. An evaluation $S, M \Downarrow \mathcal{T}$ gives a lower bound of $\text{pr}(\mathcal{T})$ for the probability of termination of a term M in the context of a stack S .

► **Definition 12.** *The probability of termination of a term M in the context of a stack S is the largest probability $\text{pr}(\mathcal{T})$ for any $\mathcal{T}$ such that $S, M \Downarrow \mathcal{T}$.*

To work with return families we use the following notation. The *sum* of two return families $\mathcal{S} + \mathcal{T}$ is defined if they have disjoint frontiers, $[\mathcal{S}] \cap [\mathcal{T}] = \emptyset$, as the union of their underlying sets $\mathcal{S} \cup \mathcal{T}$. Likewise, *inclusion* between families $\mathcal{S} \subseteq \mathcal{T}$ is inclusion as sets. We define the following notions below. On the left, $s \cdot F$ and $s \cdot \mathcal{S}$ add s as a prefix to every trace t in F , respectively in $\mathcal{S}$, and $S\mathcal{T}$ prefixes every stack in $\mathcal{T}$ with S . On the right, $\mathcal{S}|i$ restricts a family to the choice i while $\mathcal{S} \setminus i$ excludes the choice i , and $\text{stk}(\mathcal{S})$ collects the stacks in $\mathcal{S}$ as a set.

$$\begin{aligned} s \cdot F &= \{st \mid t \in F\} & \mathcal{S}|i &= \{(S, j)_t \in \mathcal{S} \mid j = i\} \\ s \cdot \mathcal{S} &= \{(S, i)_{st} \mid (S, i)_t \in \mathcal{S}\} & \mathcal{S} \setminus i &= \{(S, j)_t \in \mathcal{S} \mid j \neq i\} \\ S\mathcal{T} &= \{(ST, i)_t \mid (T, i)_t \in \mathcal{T}\} & \text{stk}(\mathcal{S}) &= \{S \mid (S, i)_t \in \mathcal{S}\} \end{aligned}$$

We will freely use the following equations.

$$[s \cdot \mathcal{S}] = s \cdot [\mathcal{S}] \quad \text{pr}(s \cdot \mathcal{S}) = \text{pr}(s) \times \text{pr}(\mathcal{S}) \quad \text{pr}(\mathcal{S} + \mathcal{T}) = \text{pr}(\mathcal{S}) + \text{pr}(\mathcal{T})$$

4 Types

Types for the FMC describe the input/output behaviour of the machine. In the deterministic case, a type τ is of the form below left, where the type vector $\bar{\sigma} = \sigma_1 \dots \sigma_k$ types the input stack on the machine, and the type vectors $\bar{\tau}_m$ type the return stack for each choice i_m . The type indicates that given an input stack $S : \bar{\sigma}$ the machine will return some choice i_m and stack $T : \bar{\tau}_m$. For the probabilistic case we replace the formal sum of the return type with a probability distribution, adding a probability p_m for each summand as below right.

$$\bar{\sigma} \Rightarrow \bar{\tau}_1 \cdot i_1 + \dots + \bar{\tau}_n \cdot i_n \quad \bar{\sigma} \Rightarrow \bar{\tau}_1 \cdot p_1 i_1 \oplus \dots \oplus \bar{\tau}_n \cdot p_n i_n$$

We will formalize the return type as *distribution type*, a family in a finite set I of choice labels, with a type vector $\bar{\tau}_i$ and a probability p_i for each $i \in I$ such that the sum probability over all p_i is at most one. It is thus a combination of a deterministic sum type (the family of type vectors $\bar{\tau}_i$) and a probability (sub-)distribution over I (the family of probabilities p_i).

► **Definition 13.** *Types, stack types, and distribution types are given as follows.*

$$\begin{aligned} \text{types} & \quad \sigma, \tau ::= \bar{\tau} \Rightarrow \delta \\ \text{stack types} & \quad \bar{\tau} ::= \tau_1 \dots \tau_n \\ \text{distribution types} & \quad \gamma, \delta ::= \{(\bar{\tau}_i, p_i)_i\}_{i \in I} \quad \text{where} \quad \sum_{i \in I} p_i \leq 1 \end{aligned}$$

We introduce the following notation to work with distribution types. The *basis* of a distribution type δ , the set of choices I over which it is defined, is $[\delta] = I$. To recover the informal notation at the start of the section, we define the *singleton* distribution type $\bar{\tau} \cdot p_i = \{(\bar{\tau}, p)_i\}$ and the *sum* $\gamma \oplus \delta = \gamma \cup \delta$ of distribution types over disjoint bases $[\gamma] \cap [\delta] = \emptyset$. The *probability* $\text{pr}(\delta)$ of a distribution type is the sum over its probabilities p_i . Below left, the notation $\bar{\sigma} \delta$ prefixes the type vector $\bar{\sigma}$ to each $\bar{\tau}$ in δ , and $q \cdot \delta$ multiplies each probability

variable	$\overline{\Gamma, x : \tau \vdash x : \tau}$	$\overline{\Gamma \vdash M : \bar{\sigma} \Rightarrow 0 \cdot \delta}$	zero
pop	$\frac{\Gamma, x : \rho \vdash M : \bar{\sigma} \Rightarrow \delta}{\Gamma \vdash \langle x \rangle. M : \rho \bar{\sigma} \Rightarrow \delta}$	$\frac{\Gamma \vdash N : \rho \quad \Gamma \vdash M : \rho \bar{\sigma} \Rightarrow \delta}{\Gamma \vdash [N]. M : \bar{\sigma} \Rightarrow \delta}$	push
choice	$\overline{\Gamma \vdash i : \bar{\sigma} \Rightarrow \bar{\sigma} \cdot 1i}$	$\frac{\Gamma \vdash M : \bar{\rho} \Rightarrow \gamma \oplus \bar{\sigma} \cdot pi \quad \Gamma \vdash N : \bar{\sigma} \Rightarrow \delta}{\Gamma \vdash M ; i \rightarrow N : \bar{\rho} \Rightarrow \gamma \sqcup (p \cdot \delta)}$	sequence
expand	$\frac{\Gamma \vdash M : \bar{\sigma} \Rightarrow \delta}{\Gamma \vdash M : \bar{\sigma} \bar{\rho} \Rightarrow \bar{\rho} \delta}$	$\frac{\Gamma \vdash M : \bar{\sigma} \Rightarrow \delta \oplus \bar{\sigma} \cdot pi \quad (p \neq 1)}{\Gamma \vdash M^i : \bar{\sigma} \Rightarrow \frac{1}{1-p} \cdot \delta}$	iterate
sample	$\frac{\Gamma, a : \varepsilon \Rightarrow \varepsilon \cdot 1\top \vdash M : \bar{\rho} \Rightarrow \gamma \quad \Gamma, a : \varepsilon \Rightarrow \varepsilon \cdot 1\perp \vdash M : \bar{\rho} \Rightarrow \delta}{\Gamma \vdash \boxed{a}. M : \bar{\rho} \Rightarrow (\frac{1}{2} \cdot \gamma) \sqcup (\frac{1}{2} \cdot \delta)}$		

■ **Figure 2** The typed probabilistic FMC

304 p_i in δ by the probability q . Below right, $\delta|I$ restricts δ to the choices in I , and $\delta \setminus I$ removes
 305 them.

$$\begin{aligned}
 306 \quad q \cdot \delta &= \{(\bar{\tau}, qp)_i \mid (\bar{\tau}, p)_i \in \delta\} & \delta|I &= \{(\bar{\tau} \cdot p)_i \in \delta \mid i \in I\} \\
 \bar{\sigma} \delta &= \{(\bar{\sigma} \bar{\tau}, p)_i \mid (\bar{\tau}, p)_i \in \delta\} & \delta \setminus I &= \{(\bar{\tau} \cdot p)_i \in \delta \mid i \notin I\}
 \end{aligned}$$

307 Finally, the *merger* $\gamma \sqcup \delta$ of two distribution types is defined if for every $i \in [\gamma] \cap [\delta]$ they
 308 have the same type vector $\bar{\tau}_i$, and is as follows.

$$309 \quad \gamma \sqcup \delta = \{(\bar{\tau}, p+q)_i \mid i \in [\gamma] \cup [\delta], (\bar{\tau}, p)_i \in \gamma, (\bar{\tau}, q)_i \in \delta\} + \gamma \setminus [\delta] + \delta \setminus [\gamma]$$

310 ► **Definition 14.** The typed probabilistic FMC ($PFMC^{\Rightarrow \oplus}$) is given by the typing rules in
 311 Figure 2, where a context $\Gamma = x_1 : \tau_1, \dots, x_n : \tau_n$ is a finite function from variables to types
 312 written as a sequence, and typing judgment $\Gamma \vdash M : \tau$ assigns a term M the type τ in the
 313 context Γ .

314 ► **Example 15.** The probabilistic sum $M \oplus N = \boxed{a}. a ; \top \rightarrow M ; \perp \rightarrow N$ is typed as follows.

$$\begin{aligned}
 &\frac{\overline{a : \varepsilon \Rightarrow \varepsilon \cdot 1\top \vdash a : \varepsilon \Rightarrow \varepsilon \cdot 1\top} \quad \overline{a : \varepsilon \Rightarrow \varepsilon \cdot 1\perp \vdash a : \varepsilon \Rightarrow \varepsilon \cdot 1\perp}}{\vdash \boxed{a}. a : \varepsilon \Rightarrow \varepsilon \cdot \frac{1}{2}\top \oplus \varepsilon \cdot \frac{1}{2}\perp} \\
 &\frac{\vdash \boxed{a}. a : \bar{\tau} \Rightarrow \bar{\tau} \cdot \frac{1}{2}\top \oplus \bar{\tau} \cdot \frac{1}{2}\perp \quad \vdash M : \bar{\tau} \Rightarrow \gamma}{\vdash \boxed{a}. a ; \top \rightarrow M : \bar{\tau} \Rightarrow \bar{\tau} \cdot \frac{1}{2}\top \oplus \frac{1}{2} \cdot \delta} \quad \vdash N : \bar{\tau} \Rightarrow \delta \\
 315 \quad &\frac{\vdash \boxed{a}. a ; \top \rightarrow M : \bar{\tau} \Rightarrow \bar{\tau} \cdot \frac{1}{2}\top \oplus \frac{1}{2} \cdot \delta \quad \vdash N : \bar{\tau} \Rightarrow \delta}{\vdash \boxed{a}. a ; \top \rightarrow M ; \perp \rightarrow N : \bar{\tau} \Rightarrow \frac{1}{2} \cdot \gamma \sqcup \frac{1}{2} \cdot \delta}
 \end{aligned}$$

316 Using this as an admissible typing rule, our running example is typed as follows.

$$\begin{aligned}
 &\frac{\vdash h : \varepsilon \Rightarrow \varepsilon \cdot 1h \quad \frac{\vdash t : \varepsilon \Rightarrow \varepsilon \cdot 1t \quad \vdash i : \varepsilon \Rightarrow \varepsilon \cdot 1i}{\vdash t \oplus i : \varepsilon \Rightarrow \varepsilon \cdot \frac{1}{2}t \oplus \varepsilon \cdot \frac{1}{2}i}}{\vdash h \oplus (t \oplus i) : \varepsilon \Rightarrow \varepsilon \cdot \frac{1}{2}h \oplus \varepsilon \cdot \frac{1}{4}t \oplus \varepsilon \cdot \frac{1}{4}i} \\
 317 \quad &\frac{\vdash h \oplus (t \oplus i) : \varepsilon \Rightarrow \varepsilon \cdot \frac{1}{2}h \oplus \varepsilon \cdot \frac{1}{4}t \oplus \varepsilon \cdot \frac{1}{4}i}{\vdash (h \oplus (t \oplus i))^i : \varepsilon \Rightarrow \varepsilon \cdot \frac{2}{3}h \oplus \varepsilon \cdot \frac{1}{3}t}
 \end{aligned}$$

► Remark 16. The type system is a conservative extension of simple types for the λ -calculus. These embed as follows, using a single default choice $\star$ (see also [12, 15, 14]). For these types, the typing rules for **variable**, **pop**, and **push** are those of the simply typed lambda-calculus.

$$\tau_1 \rightarrow \dots \rightarrow \tau_n \rightarrow o = \tau_1 \dots \tau_n \Rightarrow \varepsilon \cdot 1\star$$

► Remark 17. The type system is also a conservative extension of those of our previous work on probabilistic termination [1, 15]. Using the notation of [15], simple probabilistic types are simple types over the rationals between 0 and 1 as the set of base types, and embed as follows, where $p \in [0, 1] \cap \mathbb{Q}$ and $\star$ is again the single available choice label.

$$\tau_1 \rightarrow \dots \rightarrow \tau_n \rightarrow p = \tau_1 \dots \tau_n \Rightarrow \varepsilon \cdot p\star$$

Sequential probabilistic types [15] are the restriction of the present system to the choice $\star$.

$$\bar{\sigma} \xRightarrow{p} \bar{\tau} = \bar{\sigma} \Rightarrow \bar{\tau} \cdot p\star$$

► Remark 18. For almost-sure termination, where the probabilities for a distribution type sum to one, types may be further simplified by omitting probabilities altogether, $\tau ::= \bar{\sigma} \Rightarrow \bar{\tau}_1 \cdot i_1 + \dots + \bar{\tau}_n \cdot i_n$, by the following reasoning. Firstly, the zero typing rule is invalid, and omitted. As can be observed from the remaining typing rules, all probabilities in a typed term are then non-zero. For the **iterate** rule with a premise $M : \bar{\sigma} \Rightarrow \delta \oplus \bar{\sigma} \cdot pi$, the condition $p \neq 1$, that the chance to iterate is less than one, is then automatically fulfilled if δ is non-empty. This is the only point where the type system places a condition on the probabilities; hence they may be omitted.

Almost-Sure Termination

The intended meaning of the type system is that if $\vdash M : \varepsilon \Rightarrow \delta$ (i.e. M is closed and requires no inputs to run on the machine) then the probability of termination for M is at least $\text{pr}(\delta)$. We will establish this using a reducibility argument in the style of Tait [21], called *runnability* as it uses the machine rather than reduction. The runnability predicate directly interprets the meaning of types: a runnable term of type $M : \bar{\sigma} \Rightarrow \delta$, given a stack S of runnable terms for the types in $\bar{\sigma}$, will probabilistically evaluate to a runnable return family $\mathcal{T}$ for the type δ , with the key property that $\text{pr}(\mathcal{T}) \geq \text{pr}(\delta)$.

► **Definition 19.** The set $\text{RUN}(\tau)$ of runnable terms for a type τ is defined as the set of closed terms

$$\text{RUN}(\bar{\sigma} \Rightarrow \delta) = \{ M \mid \forall S \in \text{RUN}(\bar{\sigma}). \exists \mathcal{T} \in \text{RUN}(\delta). S, M \Downarrow \mathcal{T} \}$$

where the runnable sets for stack types $\bar{\tau}$ and distribution types δ are as follows.

$$\text{RUN}(\tau_1 \dots \tau_n) = \{ \varepsilon M_1 \dots M_n \mid M_i \in \text{RUN}(\tau_i) \}$$

$$\text{RUN}(\{(\bar{\tau}_i, p_i)\}_{i \in I}) = \{ \mathcal{T} \mid \forall i \in I. \text{stk}(\mathcal{T} \mid i) \subseteq \text{RUN}(\bar{\tau}_i) \wedge \text{pr}(\mathcal{T} \mid i) \geq p_i \}$$

To work with open terms, a *substitution map* μ is a finite function from variables to terms, applied to a term as μM which replaces every free variable x in M that is in the domain of μ with the term $\mu(x)$. For a context $\Gamma = x_1 : \tau_1, \dots, x_n : \tau_n$ the set $\text{RUN}(\Gamma)$ is a set of substitution maps μ that assigns each x_i a runnable term $M \in \text{RUN}(\tau_i)$, as follows.

$$\text{RUN}(\Gamma) = \{ \mu \mid \forall x : \tau \in \Gamma. \mu(x) \in \text{RUN}(\tau) \}$$

We are now ready to prove the *runnability lemma*, that typed terms are runnable.

358 ► **Lemma 20.** *If $\Gamma \vdash M : \tau$ and $\mu \in \text{RUN}(\Gamma)$ then $\mu M \in \text{RUN}(\tau)$.*

359 **Proof.** See the Appendix. ◀

360 The runnability lemma directly gives the main theorem of typed almost-sure termination.

361 ► **Theorem 21.** *If $\vdash M : \varepsilon \Rightarrow \delta$ then M terminates with probability at least $\text{pr}(\delta)$.*

362 **Proof.** Let $\delta = \{(\bar{\tau}_i, p_i)_{i \in I}\}$. By Lemma 20 we have $\varepsilon, M \Downarrow \mathcal{T}$ with $\mathcal{T} \in \text{RUN}(\delta)$, which
363 means that $\text{pr}(\mathcal{T}|i) \geq p_i$ for every $i \in I$, so that $\text{pr}(\mathcal{T}) \geq \text{pr}(\delta)$. ◀

364 5 Ground terms and finite Markov chains

365 We have shown that the calculus introduced in this paper enjoys all the expected properties,
366 in particular guaranteeing almost-sure termination. Several natural questions concerning the
367 expressive power of the calculus remain open. Notably, it is of interest to investigate the
368 behavior of the calculus under restrictions to low-order types, or when stronger termination
369 properties than AST, such as *positive* almost-sure termination (PAST), can be proved. In
370 this section, we consider the fragment of *ground terms*: those that involve only probabilistic
371 branching and iteration, but not input and output. They are the terms of the following form.

372 *Ground terms:* $M, N ::= i \mid M ; i \rightarrow N \mid M^i \mid \boxed{a}.a$

373 We will show the following properties: one, typing of ground terms is decidable; two, ground
374 terms readily encode as finite Markov chains; and three, finite Markov chains may be encoded
375 as ground terms. These then have the following consequences. The last two properties imply
376 that ground terms are equivalent to finite Markov chains. Next, since finite Markov chains
377 are PAST (if they are AST), also typed ground terms are PAST. Finally, combining the
378 first and last property, for the encoding of a Markov chain as a ground term, type inference
379 calculates the probability of terminating with each final state.

380 As in the introduction, we may restrict types for ground terms to sub-distributions over
381 choice labels, and abbreviate them by omitting input and return types.

382 $M : p_1 i_1 \oplus \dots \oplus p_n i_n$ abbreviates $M : \varepsilon \Rightarrow \varepsilon \cdot p_1 i_1 \oplus \dots \oplus \varepsilon \cdot p_n i_n$

383 Since ground terms are closed, the typing rules may further omit contexts. Thus restricted,
384 they are as follows.

$$385 \quad \frac{}{i : \mathbf{1}i} \quad \frac{M : \gamma \oplus pi \quad N : \delta}{M ; i \rightarrow N : \gamma \sqcup (p \cdot \delta)} \quad \frac{M : \delta \oplus pi \quad (p \neq 1)}{M^i : \frac{1}{1-p} \cdot \delta} \quad \frac{}{\boxed{a}.a : \frac{1}{2}\top \oplus \frac{1}{2}\perp} \quad \frac{}{M : \mathbf{0} \cdot \delta}$$

386 ► **Proposition 22.** *Typing for ground terms is decidable.*

387 **Proof.** Immediate by induction on the term. ◀

388 We will define Markov chains as probabilistic automata, as follows. We write $\mathcal{D}X$ for the
389 set of probability distributions over a set X , and write a distribution over $X = \{x_1, x_2, \dots\}$
390 as $\{p_1 x_1, p_2 x_2, \dots\}$.

391 ► **Definition 23.** *A Markov chain $C = (S, s, F, \delta)$ is given by a set of states S , a starting
392 state $s \in S$, a set of final states $F \subseteq S$, and a stochastic transition function $\delta : S \setminus F \rightarrow \mathcal{D}S$
393 from non-final states to states.*

394 The translation $\llbracket M \rrbracket$ from ground terms to Markov chains will take M to a chain

$$395 \quad \llbracket M \rrbracket = (S_M, s_M, F_M, \delta_M)$$

396 where internal states $S_M \setminus F_M$ represent subformulas and final states represent choices not
 397 captured by a later case construct. We consider chains modulo renaming of internal states,
 398 and when combining the translations of two terms we assume their internal states to be
 399 disjoint, but final states may overlap. To connect two chains, we use the notation $s\{t/i\}$ to
 400 rename a state, and the notation $\delta\{s/i\}$ which replaces all transitions to a (final) state i
 401 with transitions to s .

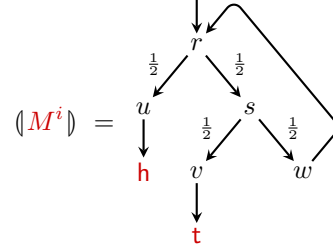
$$402 \quad \delta\{s/i\}(r) = \{ p(t\{s/i\}) \mid pt \in \delta(r) \} \quad \begin{array}{l} i\{t/i\} = t \\ s\{t/i\} = s \quad (s \neq i) \end{array}$$

403 The translation $\llbracket - \rrbracket$ is then as follows.

$$\begin{aligned} \llbracket i \rrbracket &= (\{s, i\}, s, \{i\}, s \mapsto \{i \cdot 1\}) \\ \llbracket M; i \rightarrow N \rrbracket &= (S_M \setminus \{i\} \cup S_N, s_M, F_M \setminus \{i\} \cup F_N, \delta_M\{s_N/i\} \cup \delta_N) \\ 404 \quad \llbracket M^i \rrbracket &= (S_M \setminus \{i\}, s_M, F_M \setminus \{i\}, \delta_M\{s_M/i\}) \\ \llbracket \boxed{a}.a \rrbracket &= (\{s, \top, \perp\}, s, \{\top, \perp\}, s \mapsto \{\tfrac{1}{2}\top, \tfrac{1}{2}\perp\}) \end{aligned}$$

405 **► Example 24.** For our running example term from, abbreviated as M^i where M is as below,
 406 the translation is as follows.

$$407 \quad M = \boxed{a}.a; \top \rightarrow h; \perp \rightarrow (\boxed{b}.b; \top \rightarrow t; \perp \rightarrow i)$$



$$\begin{aligned} 408 \quad \llbracket \boxed{a}.a \rrbracket &= (\{r, \top, \perp\}, r, \{\top, \perp\}, r \mapsto \{\tfrac{1}{2}\top, \tfrac{1}{2}\perp\}) \\ \llbracket \boxed{a}.a; \top \rightarrow h \rrbracket &= (\{r, u, h, \perp\}, r, \{h, \perp\}, r \mapsto \{\tfrac{1}{2}\perp, \tfrac{1}{2}u\} \quad u \mapsto \{1h\}) \\ \llbracket \boxed{b}.b; \top \rightarrow t; \perp \rightarrow i \rrbracket &= (\{s, v, w, t, i\}, s, \{t, i\}, s \mapsto \{\tfrac{1}{2}v, \tfrac{1}{2}w\} \quad v \mapsto \{1t\} \quad w \mapsto \{1i\}) \\ 409 \quad \llbracket M \rrbracket &= (\{r, s, u, v, w, h, t, i\}, r, \{h, t, i\}, \\ &\quad r \mapsto \{\tfrac{1}{2}u, \tfrac{1}{2}s\} \quad s \mapsto \{\tfrac{1}{2}v, \tfrac{1}{2}w\} \quad u \mapsto \{1h\} \quad v \mapsto \{1t\} \quad w \mapsto \{1i\}) \\ \llbracket M^i \rrbracket &= (\{r, s, u, v, w, h, t\}, r, \\ &\quad r \mapsto \{\tfrac{1}{2}u, \tfrac{1}{2}s\} \quad s \mapsto \{\tfrac{1}{2}v, \tfrac{1}{2}w\} \quad u \mapsto \{1h\} \quad v \mapsto \{1t\} \quad w \mapsto \{1r\}) \end{aligned}$$

410 In the other direction, we give an interpretation $\llbracket C \rrbracket$ of a finite Markov chain as a ground
 411 term. The translation operates as follows, using states as choice labels. Given C with starting
 412 state s and transition $s \mapsto \{p_1 t_1, \dots, p_n t_n\} \in \delta$, we remove this transition and consider the
 413 sub-chains $C(t_i)$ for $i \leq n$ with new starting state t_i , in which s becomes a final state. These
 414 are translated inductively to M_i . Then $\llbracket C \rrbracket$ is given by a term M^s , iterating on s , where M
 415 is a probabilistic sum that chooses M_i with probability p_i . We formalize this below.

416 For $C = (S, s, F, \delta)$ and $t \in S \setminus \{s\}$ let $C(t) = (S, t, F \cup \{s\}, \delta \setminus \{s\})$ where $\delta \setminus X$ is the
 417 transition function δ restricted to the domain $\text{dom}(\delta) \setminus X$. Write $p_1 M_1 \oplus \dots \oplus p_n M_n$ for the

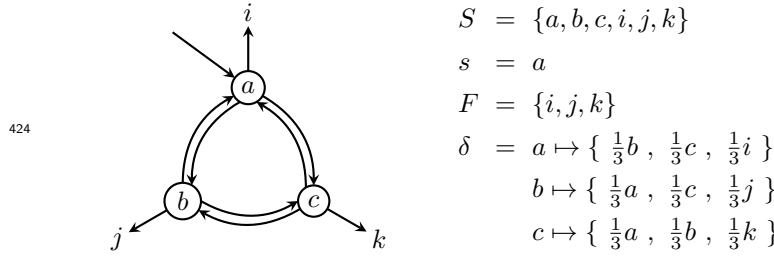
term that chooses M_i with probability p_i , given inductively below; recall that $\boxed{a}_p$ represents an unfair coin toss, given either as primitive or encoded with $\boxed{a}$ and iteration.

$$1M = M \quad pM \oplus q_1N_1 \oplus \dots \oplus q_nN_n = \boxed{a}_p \cdot a; \top \rightarrow M; \perp \rightarrow (\frac{q_1}{1-p}N_1 \oplus \dots \oplus \frac{q_n}{1-p}N_n)$$

Then the interpretation of a Markov chain $C = (S, s, F, \delta)$ as a term is as follows.

$$\llbracket C \rrbracket = \begin{cases} s & \text{if } s \in F \\ (p_1 \llbracket C(t_1) \rrbracket \oplus \dots \oplus p_n \llbracket C(t_n) \rrbracket)^s & \text{where } \delta(s) = \{p_1t_1, \dots, p_nt_n\} \end{cases}$$

► **Example 25.** Consider the following Markov chain C , where each arrow has probability $\frac{1}{3}$.



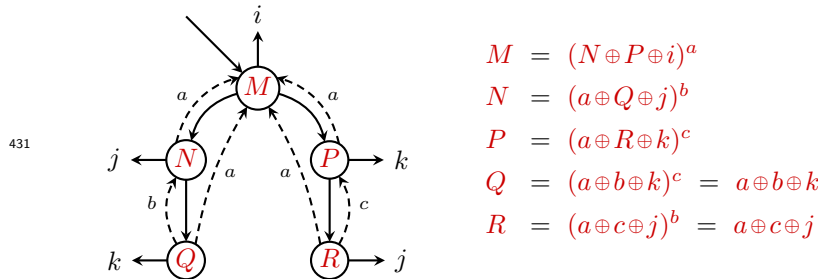
Its interpretation is as follows, writing $M \oplus N \oplus P$ for the term $\frac{1}{3}M \oplus \frac{1}{3}N \oplus \frac{1}{3}P$.

$$\begin{aligned} \llbracket C \rrbracket &= (\llbracket C(b) \rrbracket \oplus \llbracket C(c) \rrbracket \oplus i)^a \\ &= ((a \oplus \llbracket C(b)(c) \rrbracket \oplus j)^b \oplus (a \oplus \llbracket C(c)(b) \rrbracket \oplus k)^c \oplus i)^a \\ &= ((a \oplus (a \oplus b \oplus k)^c \oplus j)^b \oplus (a \oplus (a \oplus c \oplus j)^b \oplus k)^c \oplus i)^a \end{aligned}$$

This term is typed as follows.

$$\begin{aligned} (a \oplus b \oplus k)^c &: \frac{1}{3}a \oplus \frac{1}{3}b \oplus \frac{1}{3}k \\ a \oplus (a \oplus b \oplus k)^c \oplus j &: \frac{4}{9}a \oplus \frac{1}{9}b \oplus \frac{1}{9}k + \frac{1}{3}j \\ (a \oplus (a \oplus b \oplus k)^c \oplus j)^b &: \frac{1}{2}a \oplus \frac{1}{8}k \oplus \frac{3}{8}j \\ (a \oplus (a \oplus c \oplus j)^b \oplus k)^c &: \frac{1}{2}a \oplus \frac{1}{8}j \oplus \frac{3}{8}k \\ (a \oplus (a \oplus b \oplus k)^c \oplus j)^b \oplus (a \oplus (a \oplus c \oplus j)^b \oplus k)^c \oplus i &: \frac{1}{3}a \oplus \frac{1}{6}k \oplus \frac{1}{6}j \oplus \frac{1}{3}i \\ ((a \oplus (a \oplus b \oplus k)^c \oplus j)^b \oplus (a \oplus (a \oplus c \oplus j)^b \oplus k)^c \oplus i)^a &: \frac{1}{4}k \oplus \frac{1}{4}j \oplus \frac{1}{2}i \end{aligned}$$

The interpretation may be illustrated as follows, where the solid edges represent the tree-shape of the probabilistic sums, while the dashed edges represent the loops.



As the illustration suggests, the formalism may express any probabilistic tree with back-pointers. It may also express directed acyclic graphs, which yields a smaller interpretation of the Markov chain above, though still exponential, in the following term.

$$(b \oplus c \oplus i; b \rightarrow a \oplus c \oplus j; c \rightarrow (a \oplus b \oplus k; b \rightarrow a \oplus c \oplus j)^c)^a$$

The types work out the same way.

$$\begin{aligned} b \oplus c \oplus i & : \frac{1}{3}b \oplus \frac{1}{3}c \oplus \frac{1}{3}i \\ b \oplus c \oplus i; b \rightarrow a \oplus c \oplus j & : \frac{1}{9}a \oplus \frac{4}{9}c \oplus \frac{1}{3}i \oplus \frac{1}{9}j \\ (a \oplus b \oplus k; b \rightarrow a \oplus c \oplus j)^c & : \frac{1}{2}a \oplus \frac{1}{8}j \oplus \frac{3}{8}k \\ b \oplus c \oplus i; b \rightarrow a \oplus c \oplus j; c \rightarrow (a \oplus b \oplus k; b \rightarrow a \oplus c \oplus j)^c & : \frac{1}{3}a \oplus \frac{1}{3}i \oplus \frac{1}{6}j \oplus \frac{1}{6}k \\ (b \oplus c \oplus i; b \rightarrow a \oplus c \oplus j; c \rightarrow (a \oplus b \oplus k; b \rightarrow a \oplus c \oplus j)^c)^a & : \frac{1}{2}i \oplus \frac{1}{4}j \oplus \frac{1}{4}k \end{aligned}$$

The translations $\llbracket - \rrbracket$ and $\llbracket - \rrbracket$ above demonstrate the following.

► **Proposition 26.** *Ground terms of the PFMC are equivalent to finite Markov chains.*

Markov chains are PAST if they are AST, i.e. if any path may eventually reach a final state. For typed ground terms, which are AST, we then have the following.

► **Proposition 27.** *Typed ground terms of the $PFMC^{\rightarrow \oplus}$ are PAST.*

Combining type inference for ground terms (Proposition 22) and the translation $\llbracket - \rrbracket$, we have the following result.

► **Proposition 28.** *Encoding and typing a finite Markov chain computes its return distribution: if $\llbracket C \rrbracket : p_1 s_1 \oplus \dots \oplus p_n s_n$ then C terminates in the state s_i with probability p_i .*

Proof. By construction. ◀

6 Moving past PAST

Further exploring the expressive power of the calculus, in this section we demonstrate that typed terms in general are NAST (i.e. AST, but not PAST). We will give an example of a term with infinite expected running time, despite terminating with probability one. Our example relies crucially on higher-order functions.

We will use the default choice $\star$ as imperative *skip*, and its case as sequential composition, $M; N = M; \star \rightarrow N$. Consider the following term, for a given M .

$$[M].((\langle f \rangle. [f; f]. i) \oplus \star)^i$$

With probability $\frac{1}{2}$ it composes the term on the stack with itself and iterates, or it terminates. Evaluating this term on the machine gives the following return family, for the sequence of terms M_n given by $M_0 = M$ and $M_{n+1} = M_n; M_n$.

$$\{(M_n, \star)_{\top \perp} \mid n \in \mathbb{N}\} = \{(M, \star)_{\perp}, (M; M, \star)_{\top \perp}, ((M; M); (M; M), \star)_{\top \top \perp}, \dots\}$$

We may then evaluate the term M_n left on the stack by post-composing with $\langle g \rangle.g$, which pops and runs it, possibly using further base values further down on the stack. If M terminates successfully with $\star$ in m steps, M_n takes $m \times 2^n$ steps. For example, assuming

a natural numbers type $\mathbb{N}$ with primitives zero $0 : \mathbb{N}$ and addition $+$: $\mathbb{N} \times \mathbb{N} \Rightarrow \mathbb{N} \cdot 1\star$, we let M be the successor function $[1]. + : \mathbb{N} \Rightarrow \mathbb{N} \cdot 1\star$. The term below left then gives the return family below right, taking $m \times 2^n$ machine steps (for some small constant m) to compute the number $2^n : \mathbb{N}$ from zero and successor. Since the probability of returning 2^n is $\frac{1}{2^{n+1}}$, the expected evaluation time on the machine is infinite: the term is not PAST.

$$[0]. [[1]. +]. ((\langle f \rangle. [f; f]. i) \oplus \star)^i ; \langle g \rangle. g : \varepsilon \Rightarrow \mathbb{N} \cdot 1\star \quad \{(2^n, \star)_{\top^n \perp} \mid n \in \mathbb{N}\}$$

The term is typed: for any $\bar{\tau}$, if M is typed $M : \bar{\tau} \Rightarrow \bar{\tau} \cdot 1\star$ then we have $M^n : \bar{\tau} \Rightarrow \bar{\tau} \cdot 1\star$, such as in the case above where $\bar{\tau} = \mathbb{N}$. For the loop we then have the following.

$$\frac{\vdash ((\langle f \rangle. [f; f]. i) \oplus \star : \sigma \Rightarrow \sigma \cdot \frac{1}{2}i \oplus \sigma \cdot \frac{1}{2}\star)}{\vdash (((\langle f \rangle. [f; f]. i) \oplus \star)^i : \sigma \Rightarrow \sigma \cdot 1\star)} \quad \text{where } \sigma = \bar{\tau} \Rightarrow \bar{\tau} \cdot 1\star$$

The term $\langle g \rangle. g : (\bar{\tau} \Rightarrow \bar{\tau} \cdot 1\star) \bar{\tau} \Rightarrow \bar{\tau} \cdot 1\star$ takes $M^n : \bar{\tau} \Rightarrow \bar{\tau} \cdot 1\star$ off the stack and evaluates it with an appropriate remaining stack of type $\bar{\tau}$. The existence of a typed term that is not PAST then demonstrates the following.

► **Proposition 29.** *The typed probabilistic FMC is not positive almost-surely terminating.*

7 Conclusion

We have demonstrated a probabilistic interpretation of the branching and iteration in the FMC to give a type system for almost-sure termination, the PFMC $\Rightarrow^\oplus$. We believe this to be a natural account of almost-sure termination for the following reasons.

The central idea, with which we started the introduction, is to take the familiar typed iterator $\text{iter } f$ and make it probabilistic. Immediately, this gives an account of typed almost-sure termination. We explored the consequences of this idea within the *control* extension of the FMC, consisting of the *choice*, *case*, and *loop* constructs. In itself, this forms a minimal basis for imperative control flow, modelling conditionals, constants, case switches, exception handling, and iteration, all on simple abstract machine with a single continuation stack, both typed and untyped—unlike the iterator $\text{iter } f$, which must be typed. Combined with our probabilistic generator, the control constructs form the *ground terms* of the PFMC, which we showed to be equivalent to finite Markov chains, where typing computes the return distribution of the chain.

The full PFMC is then an orthogonal and conservative combination of the control extension with the λ -calculus and the probabilistic generator. Each fragment operates on an independent stack of the extended Krivine machine, simply by pushing and popping. The type system is a conservative extension of simple types for the λ -calculus, where each fragment contributes a natural component: the control fragment, a formal sum of return values indexed by *choice* labels; the generator, a probability (sub-)distribution over these choice labels. The combination of probabilistic almost-sure termination and higher-order functions then gives the calculus expressive power beyond positive AST.

In future work, we intend to further analyse the expressive power of the calculus and fragments of it. At present, being an extension of simple types, it is clear that the expressive power of the typed calculus for real algorithms is limited. Addressing this, we expect that the type system will generalize to dependent types and intersection types along standard lines. This is not idle speculation: the principle was already demonstrated for intersection types for higher-order store in the FMC [13]. A dependently typed generalisation would be akin to Hoare logic, and might facilitate reasoning about probabilistic termination in theorem provers and give natural ways to include termination evidence in typed imperative programming languages.

References

- 1 Melissa Antonelli, Ugo Dal Lago, and Paolo Pistone. Curry and Howard meet Borel. In *Proceedings of the 37th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS'22)*, pages 45:1–45:13. ACM, 2022. doi:10.1145/3531130.3533361.
- 2 Martin Avanzini, Ugo Dal Lago, and Alexis Ghyselen. Type-based complexity analysis of probabilistic functional programs. In *34th Annual ACM/IEEE Symposium on Logic in Computer Science, LICS*, pages 1–13. IEEE, 2019. doi:10.1109/LICS.2019.8785725.
- 3 Flavien Breuvar, Ugo Dal Lago, and Agathe Herrou. On higher-order probabilistic subrecursion. *Log. Methods Comput. Sci.*, 17(4), 2021. doi:10.46298/LMCS-17(4:25)2021.
- 4 Ugo Dal Lago, Claudia Faggian, and Simona Ronchi Della Rocca. Intersection types and (positive) almost-sure termination. *Proceedings of the ACM on Programming Languages*, 5(POPL):1–32, 2021. doi:10.1145/3434313.
- 5 Ugo Dal Lago and Charles Grellois. Probabilistic termination by monadic affine sized typing. *ACM Trans. Program. Lang. Syst.*, 41(2):10:1–10:65, 2019. doi:10.1145/3293605.
- 6 Ugo Dal Lago, Giulio Guerrieri, and Willem Heijltjes. Decomposing probabilistic lambda-calculi. In Jean Goubault-Larrecq and Barbara König, editors, *Foundations of Software Science and Computation Structures*, volume 12077 of *LNCS*, pages 136–156, Cham, 2020. Springer International Publishing. doi:10.1007/978-3-030-45231-5_8.
- 7 Ugo Dal Lago and Luca Padovani. On the almost-sure termination of binary sessions. In Alessandro Bruni, Alberto Momigliano, Matteo Pradella, Matteo Rossi, and James Cheney, editors, *Proceedings of the 26th International Symposium on Principles and Practice of Declarative Programming, PPDP*, pages 9:1–9:12. ACM, 2024. doi:10.1145/3678232.3678239.
- 8 Ankush Das, Di Wang, and Jan Hoffmann. Probabilistic resource-aware session types. *Proc. ACM Program. Lang.*, 7(POPL):1925–1956, 2023. doi:10.1145/3571259.
- 9 Javier Esparza, Antonín Kucera, and Richard Mayr. Model checking probabilistic pushdown automata. In *19th IEEE Symposium on Logic in Computer Science (LICS)*, pages 12–21. IEEE Computer Society, 2004. doi:10.1109/LICS.2004.1319596.
- 10 Kousha Etessami and Mihalis Yannakakis. Recursive markov chains, stochastic grammars, and monotone systems of nonlinear equations. *J. ACM*, 56(1):1:1–1:66, 2009. doi:10.1145/1462153.1462154.
- 11 Luis María Ferrer Fioriti and Holger Hermanns. Probabilistic termination: Soundness, completeness, and compositionality. In Sriram K. Rajamani and David Walker, editors, *Proceedings of the 42nd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL*, pages 489–501. ACM, 2015. doi:10.1145/2676726.2677001.
- 12 Willem Heijltjes. The Functional Machine Calculus. *Electronic Notes in Theoretical Informatics and Computer Science*, Volume 1 - Proceedings of MFPS XXXVIII, 2023. URL: <https://entics.episciences.org/10513>, doi:10.46298/entics.10513.
- 13 Willem Heijltjes. Quantitative types for the Functional Machine Calculus. In Maribel Fernández, editor, *10th International Conference on Formal Structures for Computation and Deduction (FSCD)*, volume 337 of *LIPICs*, pages 24:1–24:20. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025. doi:10.4230/LIPICs.FSCD.2025.24.
- 14 Willem Heijltjes. The Functional Machine Calculus III: Control. *Electronic Notes in Theoretical Informatics and Computer Science*, Volume 5 - Proceedings of MFPS XLI, 2025. URL: <https://entics.episciences.org/16682>, doi:10.46298/entics.16682.
- 15 Willem Heijltjes and Georgina Majury. Simple types for probabilistic termination. In Jörg Endrullis and Sylvain Schmitz, editors, *33rd EACSL Annual Conference on Computer Science Logic (CSL)*, volume 326 of *LIPICs*, pages 31:1–31:22. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025. doi:10.4230/LIPICs.CSL.2025.31.
- 16 Naoki Kobayashi, Ugo Dal Lago, and Charles Grellois. On the termination problem for probabilistic higher-order recursive programs. *Log. Methods Comput. Sci.*, 16(4), 2020. doi:10.23638/LMCS-16(4:2)2020.

- 558 17 Jean-Louis Krivine. A call-by-name lambda-calculus machine. *Higher-Order and Symbolic*
559 *Computation*, 20:199–207, 2007. doi:10.1007/s10990-007-9018-9.
- 560 18 Paul Blain Levy. *Call-by-push-value: A functional/imperative synthesis*, volume 2 of *Semantic*
561 *Structures in Computation*. Springer Netherlands, 2003. doi:10.1007/978-94-007-0954-6.
- 562 19 Annabelle McIver, Carroll Morgan, Benjamin Lucien Kaminski, and Joost-Pieter Katoen. A
563 new proof rule for almost-sure termination. *Proc. ACM Program. Lang.*, 2(POPL):33:1–33:28,
564 2018. doi:10.1145/3158121.
- 565 20 Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–
566 92, 1991. doi:10.1016/0890-5401(91)90052-4.
- 567 21 W.W. Tait. Intensional interpretations of functionals of finite type I. *The Journal of Symbolic*
568 *Logic*, 32(2):198–212, 1967. doi:10.2307/2271658.

569 **A** Appendix

570 Running the machine

571 ► **Example 30.** Below we give a successful run of the machine for our running example term,
572 with the definition of $M \oplus N$ unfolded to its primitives, and the trace $\perp \perp \top$.

573	$M = (\mathbf{h} \oplus (\mathbf{t} \oplus i)) = (\boxed{a}.a; \top \rightarrow \mathbf{h}; \perp \rightarrow (\boxed{b}.b; \top \rightarrow \mathbf{t}; \perp \rightarrow i))^i$	
574	$(\perp \perp \top, \varepsilon, (\boxed{a}.a; \top \rightarrow \mathbf{h}; \perp \rightarrow (\boxed{b}.b; \top \rightarrow \mathbf{t}; \perp \rightarrow i))^i, \varepsilon)$ $(\perp \perp \top, \varepsilon, \boxed{a}.a; \top \rightarrow \mathbf{h}; \perp \rightarrow (\boxed{b}.b; \top \rightarrow \mathbf{t}; \perp \rightarrow i), (i \rightarrow M))$ $(\perp \perp \top, \varepsilon, \boxed{a}.a; \top \rightarrow \mathbf{h}, (\perp \rightarrow (\boxed{b}.b; \top \rightarrow \mathbf{t}; \perp \rightarrow i)) (i \rightarrow M))$ $(\perp \perp \top, \varepsilon, \boxed{a}.a, (\top \rightarrow \mathbf{h}) (\perp \rightarrow (\boxed{b}.b; \top \rightarrow \mathbf{t}; \perp \rightarrow i)) (i \rightarrow M))$ $(\perp \top, \varepsilon, \perp, (\top \rightarrow \mathbf{h}) (\perp \rightarrow (\boxed{b}.b; \top \rightarrow \mathbf{t}; \perp \rightarrow i)) (i \rightarrow M))$ $(\perp \top, \varepsilon, \perp, (\perp \rightarrow (\boxed{b}.b; \top \rightarrow \mathbf{t}; \perp \rightarrow i)) (i \rightarrow M))$ $(\perp \top, \varepsilon, \boxed{b}.b; \top \rightarrow \mathbf{t}; \perp \rightarrow i, (i \rightarrow M))$ $(\perp \top, \varepsilon, \boxed{b}.b; \top \rightarrow \mathbf{t}, (\perp \rightarrow i) (i \rightarrow M))$ $(\perp \top, \varepsilon, \boxed{b}.b, (\top \rightarrow \mathbf{t}) (\perp \rightarrow i) (i \rightarrow M))$	
575	$(\top, \varepsilon, \perp, (\top \rightarrow \mathbf{t}) (\perp \rightarrow i) (i \rightarrow M))$ $(\top, \varepsilon, \perp, (\perp \rightarrow i) (i \rightarrow M))$ $(\top, \varepsilon, i, (i \rightarrow M))$ $(\top, \varepsilon, (\boxed{a}.a; \top \rightarrow \mathbf{h}; \perp \rightarrow (\boxed{b}.b; \top \rightarrow \mathbf{t}; \perp \rightarrow i))^i, \varepsilon)$ $(\top, \varepsilon, \boxed{a}.a; \top \rightarrow \mathbf{h}; \perp \rightarrow (\boxed{b}.b; \top \rightarrow \mathbf{t}; \perp \rightarrow i), (i \rightarrow M))$ $(\top, \varepsilon, \boxed{a}.a; \top \rightarrow \mathbf{h}, (\perp \rightarrow (\boxed{b}.b; \top \rightarrow \mathbf{t}; \perp \rightarrow i)) (i \rightarrow M))$ $(\top, \varepsilon, \boxed{a}.a, (\top \rightarrow \mathbf{h}) (\perp \rightarrow (\boxed{b}.b; \top \rightarrow \mathbf{t}; \perp \rightarrow i)) (i \rightarrow M))$ $(\varepsilon, \varepsilon, \top, (\top \rightarrow \mathbf{h}) (\perp \rightarrow (\boxed{b}.b; \top \rightarrow \mathbf{t}; \perp \rightarrow i)) (i \rightarrow M))$ $(\varepsilon, \varepsilon, \mathbf{h}, (\perp \rightarrow (\boxed{b}.b; \top \rightarrow \mathbf{t}; \perp \rightarrow i)) (i \rightarrow M))$ $(\varepsilon, \varepsilon, \mathbf{h}, (i \rightarrow M))$ $(\varepsilon, \varepsilon, \mathbf{h}, \varepsilon)$	

576 Proving the runnability lemma

577 Here we prove Lemma 20, the runnability lemma. Recall the statement of the Lemma:

578 **Runnability lemma:** *If $\Gamma \vdash M : \tau$ and $\mu \in \text{RUN}(\Gamma)$ then $\mu M \in \text{RUN}(\tau)$.*

579 **Proof.** By induction on the typing derivation.

580 Variable:

$$\Gamma, x : \tau \vdash x : \tau$$

582 By the definition of $\text{RUN}(\Gamma)$ we have $\mu x \in \text{RUN}(\tau)$.

583 *Choice:*

584

$$\overline{\Gamma \vdash i : \overline{\sigma} \Rightarrow \overline{\sigma} \cdot 1i}$$

Let $S \in \text{RUN}(\overline{\sigma})$ and note that $\mu i = i$ for any μ . We have the evaluation $\varepsilon, S, i \Downarrow S, i$ for the empty trace. Then for the frontier $\{\varepsilon\}$ we have $S, i \Downarrow \{(S, i)_\varepsilon\}$, with the singleton return family of S and i for the empty trace. Since $S \in \text{RUN}(\overline{\sigma})$ and $p(\varepsilon) = 1$ it follows that $\{(S, i)_\varepsilon\} \in \text{RUN}(\overline{\sigma} \cdot 1i)$.

589 ■ *Zero*

$$590 \quad \frac{}{\Gamma \vdash M : \bar{\sigma} \Rightarrow 0 \cdot \delta}$$

591 For any S and μ we have $S, \mu M \Downarrow \emptyset$. Since $\emptyset \subseteq \text{RUN}(\bar{\tau})$ for any $\bar{\tau}$ and $\text{pr}(\emptyset|i) \geq 0$ for
 592 any i we have $\emptyset \in \text{RUN}(0 \cdot \delta)$.

593 ■ *Expand:*

$$594 \quad \frac{\Gamma \vdash M : \bar{\sigma} \Rightarrow \delta}{\Gamma \vdash M : \bar{\sigma} \bar{\rho} \Rightarrow \bar{\rho} \delta}$$

595 Let $R \in \text{RUN}(\bar{\rho})$, $S \in \text{RUN}(\bar{\sigma})$, and $\mu \in \text{RUN}(\Gamma)$. The inductive hypothesis gives the
 596 evaluation $S, M \Downarrow \mathcal{T}$. For each $(T, i)_t \in \mathcal{T}$ we have the evaluation below left. By induction
 597 on the evaluation relation $\Downarrow$ we get the evaluation below right.

$$598 \quad t, S, \mu M \Downarrow T, i \quad t, RS, \mu M \Downarrow RT, i$$

599 This gives $RS, \mu M \Downarrow R\mathcal{T}$. Since $R \in \text{RUN}(\bar{\rho})$ and $\text{stk}(\mathcal{T}|i) \subseteq \text{RUN}(\bar{\tau}_i)$, where $\bar{\tau}_i$ is the
 600 type for i in δ , we have $\text{stk}(R\mathcal{T}|i) \subseteq \text{RUN}(\bar{\rho} \bar{\tau}_i)$. Since also $\text{pr}(R\mathcal{T}|i) = \text{pr}(\mathcal{T}|i) \geq p_i$ we
 601 have $R\mathcal{T} \in \text{RUN}(\bar{\rho} \delta)$.

602 ■ *Push:*

$$603 \quad \frac{\Gamma \vdash N : \rho \quad \Gamma \vdash M : \rho \bar{\sigma} \Rightarrow \delta}{\Gamma \vdash [N]. M : \bar{\sigma} \Rightarrow \delta}$$

604 Let $S \in \text{RUN}(\bar{\sigma})$ and $\mu \in \text{RUN}(\Gamma)$. By induction we have $\mu N \in \text{RUN}(\rho)$ and $\mu M \in$
 605 $\text{RUN}(\rho \bar{\sigma} \Rightarrow \delta)$, and for the latter the evaluation $S \mu N, \mu M \Downarrow \mathcal{T}$. Then for every $(T, i)_t \in \mathcal{T}$
 606 we have the evaluation below left, from which we construct the derivation below right.

$$607 \quad t, S \mu N, \mu M \Downarrow T, i \quad \frac{t, S \mu N, \mu M \Downarrow T, i}{t, S, \mu([N]. M) \Downarrow T, i}^{\text{push}}$$

608 Then $S, \mu([N]. M) \Downarrow \mathcal{T}$ and $\mu([N]. M) \in \text{RUN}(\bar{\sigma} \Rightarrow \delta)$.

609 ■ *Pop:*

$$610 \quad \frac{\Gamma, x : \rho \vdash M : \bar{\sigma} \Rightarrow \delta}{\Gamma \vdash \langle x \rangle. M : \rho \bar{\sigma} \Rightarrow \delta}$$

611 Let $S \in \text{RUN}(\bar{\sigma})$, $N \in \text{RUN}(\rho)$, and $\mu \in \text{RUN}(\Gamma)$. Let $\mu' = \mu\{N/x\}$, the substitution
 612 map that sends x to N and every other variable y to μy , so that $\mu' \in \text{RUN}(\Gamma, x : \rho)$. By
 613 induction we have $S, \mu' M \Downarrow \mathcal{T}$, given by the evaluation below for every $(T, i)_t \in \mathcal{T}$, from
 614 which we construct the evaluation below right.

$$615 \quad t, S, \mu' M \Downarrow T, i \quad \frac{t, S, \mu' M \Downarrow T, i}{t, S N, \mu(\langle x \rangle. M) \Downarrow T, i}^{\text{pop}}$$

616 Then $S N, \mu(\langle x \rangle. M) \Downarrow \mathcal{T}$ and $\mu(\langle x \rangle. M) \in \text{RUN}(\rho \bar{\sigma} \Rightarrow \delta)$.

617 ■ *Sample:*

$$618 \quad \frac{\Gamma, a : \varepsilon \Rightarrow \varepsilon \cdot 1 \top \vdash M : \bar{\rho} \Rightarrow \gamma \quad \Gamma, a : \varepsilon \Rightarrow \varepsilon \cdot 1 \perp \vdash M : \bar{\rho} \Rightarrow \delta}{\Gamma \vdash \boxed{a}. M : \bar{\rho} \Rightarrow (\frac{1}{2} \cdot \gamma) \sqcup (\frac{1}{2} \cdot \delta)}$$

619 Let $R \in \text{RUN}(R)$ and $\mu \in \text{RUN}(\Gamma)$. Define $\mu_{\top} = \mu\{\top/a\}$ and $\mu_{\perp} = \mu\{\perp/a\}$ so that
 620 $\mu_{\top} \in \text{RUN}(\Gamma, a : \varepsilon \Rightarrow \varepsilon \cdot 1 \top)$ and $\mu_{\perp} \in \text{RUN}(\Gamma, a : \varepsilon \Rightarrow \varepsilon \cdot 1 \perp)$. By induction we have return

families $\mathcal{S}$ and $\mathcal{T}$ where $R, \mu_{\top} M \Downarrow \mathcal{S}$ and $R, \mu_{\perp} M \Downarrow \mathcal{T}$, given by the following evaluations for every $(S, i)_s \in \mathcal{S}$ and $(T, j)_t \in \mathcal{T}$.

$$s, R, \mu_{\top} M \Downarrow S, i \quad t, R, \mu_{\perp} M \Downarrow T, j$$

For these we have the following derivations.

$$\frac{s, R, \mu_{\top} M \Downarrow S, i}{\top s, R, \mu(\boxed{a}. M) \Downarrow S, i} \text{smpl} \quad \frac{t, R, \mu_{\perp} M \Downarrow T, j}{\perp t, R, \mu(\boxed{a}. M) \Downarrow T, j} \text{smpl}$$

Then for $\mathcal{U} = (\top \cdot \mathcal{S}) \cup (\perp \cdot \mathcal{T})$ we have $S, \mu(\boxed{a}. M) \Downarrow \mathcal{U}$. It remains to show that $\mathcal{U} \in \text{RUN}(\frac{1}{2} \cdot \gamma \sqcup \frac{1}{2} \cdot \delta)$. For any indexed pair in $\frac{1}{2} \cdot \gamma \sqcup \frac{1}{2} \cdot \delta$, by the definition of $(\sqcup)$ we have three cases.

- $(\bar{\tau}, \frac{1}{2}p + \frac{1}{2}q)_i$ where $i \in [\gamma] \cap [\delta]$, $(\bar{\tau} \cdot p)_i \in \gamma$ and $(\bar{\tau} \cdot q)_i \in \delta$. Then since $\text{stk}(\mathcal{S}|i) \subseteq \text{RUN}(\bar{\tau})$ and $\text{stk}(\mathcal{T}|i) \subseteq \text{RUN}(\bar{\tau})$ we have $\text{stk}(\mathcal{U}|i) \subseteq \text{RUN}(\bar{\tau})$, and since $\text{pr}(\mathcal{S}|i) \geq p$ and $\text{pr}(\mathcal{T}|i) \geq q$ we have $\text{pr}(\mathcal{U}|i) = \text{pr}(\top \cdot \mathcal{S}|i) + \text{pr}(\perp \cdot \mathcal{T}|i) \geq \frac{1}{2}p + \frac{1}{2}q$.
- $(\bar{\tau}, \frac{1}{2}p)_i$ in $\frac{1}{2} \cdot \gamma \setminus [\delta]$. Since $\text{stk}(\mathcal{S}|i) \subseteq \text{RUN}(\bar{\tau})$ we have $\text{stk}(\mathcal{U}|i) \subseteq \text{RUN}(\bar{\tau})$, and since $\text{pr}(\mathcal{S}|i) \geq p$ we have $\text{pr}(\mathcal{U}|i) = \text{pr}(\top \cdot \mathcal{S}|i) \geq \frac{1}{2}p$.
- The case for $\frac{1}{2} \cdot \delta \setminus [\gamma]$ is analogous to the previous.

■ *Sequence:*

$$\frac{\Gamma \vdash M : \bar{p} \Rightarrow \gamma \oplus \bar{\sigma} \cdot pi \quad \Gamma \vdash N : \bar{\sigma} \Rightarrow \delta}{\Gamma \vdash M ; i \rightarrow N : \bar{p} \Rightarrow \gamma \sqcup p \cdot \delta}$$

Let $R \in \text{RUN}(\bar{p})$ and $\mu \in \text{RUN}(\Gamma)$. By induction we have $\mu M \in \text{RUN}(\bar{p} \Rightarrow \gamma \oplus \bar{\sigma} \cdot pi)$ and $R, \mu M \Downarrow S$. Let this be given by an evaluation for each $(S, j)_s \in \mathcal{S}$ of the form below left. For each $(S, j)_s$ where $j = i$ we have $S \in \text{RUN}(\bar{\sigma})$, and by induction on N a return family $\mathcal{T}_s$ such that $S, \mu N \Downarrow \mathcal{T}_s$. Let this be given by the evaluation below right for each $(T, k)_t \in \mathcal{T}_s$.

$$s, R, \mu M \Downarrow S, j \quad t, S, \mu N \Downarrow T, k$$

For each $(S, j)_s$, if $j \neq i$ we construct the derivation below left, and if $j = i$ then for each $(T, k)_t \in \mathcal{T}_s$ we construct that below right.

$$\frac{s, R, \mu M \Downarrow S, j}{s, R, \mu(M ; i \rightarrow N) \Downarrow S, j} \text{rej} \quad \frac{s, R, \mu M \Downarrow S, j \quad t, S, \mu N \Downarrow T, k}{st, R, \mu(M ; i \rightarrow N) \Downarrow T, k} \text{sel}$$

Then we have $R, \mu(M ; i \rightarrow N) \Downarrow \mathcal{T}$ where $\mathcal{T}$ is given as follows.

$$\mathcal{T} = (\mathcal{S} \setminus i) \cup \bigcup_{s \in [\mathcal{S}|i]} s \cdot \mathcal{T}_s$$

It remains to show that $\mathcal{T} \in \text{RUN}(\gamma \sqcup p \cdot \delta)$.

- First, for the return types, for all $j \in [\gamma] \cup [\delta]$ we need that $\text{stk}(\mathcal{T}|j) \subseteq \text{RUN}(\bar{\tau})$ where $\bar{\tau}$ is the type for j in γ and δ (the definition of $(\sqcup)$ requires that they agree). By definition any indexed pair in $\mathcal{T}$ is either $(S, j)_s \in \mathcal{S} \setminus i$ or $(T, k)_t \in \mathcal{T}_s$. The former case follows since $\mathcal{S} \in \text{RUN}(\gamma)$ and so $\text{stk}(\mathcal{S}|j) \subseteq \text{RUN}(\bar{\tau})$ and $S \in \text{RUN}(\bar{\tau})$. The latter case follows since $\mathcal{T}_s \in \text{RUN}(\delta)$, so that $\text{stk}(\mathcal{S}|j) \subseteq \text{RUN}(\bar{\tau})$ and $T \in \text{RUN}(\bar{\tau})$.

- 654 $\dashv$ Second, for the probabilities, for all $j \in [\gamma] \cup [\delta]$ it must be shown that $\text{pr}(\mathcal{T}|j) \geq$
 655 $p_j + pq_j$ where p_j is the probability of j in γ and q_j that in δ . We compute as follows.

$$\begin{aligned}
 \text{pr}(\mathcal{T}|j) &= \text{pr}((\mathcal{S} \setminus i) \cup \bigcup_{s \in [\mathcal{S}|i]} s \cdot \mathcal{T}_s | j) \\
 &= \text{pr}((\mathcal{S} \setminus i) | j) + \sum_{s \in [\mathcal{S}|i]} (\text{pr}(s) \times \text{pr}(\mathcal{T}_s | j)) \\
 656 &\geq p_j + \sum_{s \in [\mathcal{S}|i]} (\text{pr}(s) \times q_j) \\
 &\geq p_j + pq_j
 \end{aligned}$$

657 For the third step (the first inequality), since $\mathcal{S} \setminus i \in \text{RUN}(\gamma)$ we have $\text{pr}(\mathcal{S} | j) \geq p_j$ for
 658 all $j \in [\gamma]$. For the last step, since $\mathcal{T}_s \in \text{RUN}(\delta)$ for all s we have $\text{pr}(\mathcal{T}_s | j) \geq q_j$ for all
 659 $j \in [\delta]$. Then since $\mathcal{S} | i \in \text{RUN}(\bar{\sigma} \cdot pi)$ we have $\text{pr}(\mathcal{S} | i) \geq p$.

660 It follows that $\mathcal{T} \in \text{RUN}(\gamma \sqcup p \cdot \delta)$ and hence $\mu(M; i \rightarrow N) \in \text{RUN}(\bar{p} \Rightarrow \gamma \sqcup p \cdot \delta)$.

661 $\dashv$ *Iterate:*

$$\frac{\Gamma \vdash M : \bar{\sigma} \Rightarrow \delta + \bar{\sigma} \cdot pi \quad (p \neq 1)}{\Gamma \vdash M^i : \bar{\sigma} \Rightarrow \frac{1}{1-p} \cdot \delta}$$

663 Let $S \in \text{RUN}(\bar{\sigma})$ and $\mu \in \text{RUN}(\Gamma)$. We construct a series of return families $\mathcal{T}_n$ for all
 664 $n \in \mathbb{N}^+$ by induction on n , and show the following.

$$\mathcal{T}_n \in \text{RUN}((q_n \cdot \delta) + \bar{\sigma} \cdot p^n i) \quad \text{where} \quad q_n = 1 + p + p^2 + p^3 + \dots + p^{n-1}$$

- 666 $\dashv$ The case $\mathcal{T}_1$ is given by induction on M by $S, \mu M \Downarrow \mathcal{T}_1$.
 667 $\dashv$ For the case $\mathcal{T}_{n+1}$, for each $(R, i)_r \in \mathcal{T}_n$, by induction on M let $R, \mu M \Downarrow \mathcal{T}_r$ and define
 668 $\mathcal{T}_{n+1}$ as follows.

$$\mathcal{T}_{n+1} = (\mathcal{T}_n \setminus i) \cup \bigcup_{r \in [\mathcal{T}_n | i]} r \cdot \mathcal{T}_r$$

670 By induction we have $\mathcal{T}_n \in \text{RUN}(q_n \cdot \delta + \bar{\sigma} \cdot p^n i)$. By similar reasoning to the previous
 671 *sequencing* case we have the following.

$$\mathcal{T}_{n+1} \in \text{RUN}(q_n \cdot \delta + p^n \cdot (\delta + \bar{\sigma} \cdot pi)) = \text{RUN}(q_{n+1} \cdot \delta + \bar{\sigma} \cdot p^{n+1} i)$$

673 Observe that by definition $\mathcal{T}_n \setminus i \subseteq \mathcal{T}_{n+1} \setminus i$. Then we define $\mathcal{T}$ as the following supremum.

$$\mathcal{T} = \sup_{n \in \mathbb{N}^+} \mathcal{T}_n \setminus i$$

675 It remains to show that $S, \mu(M^i) \Downarrow \mathcal{T}$ and $\mathcal{T} \in \text{RUN}(\frac{1}{1-p} \cdot \delta)$.

- 676 $\dashv$ If $(T, j)_t \in \mathcal{T}$, then by definition $(T, j)_t \in \mathcal{T}_n \setminus i$ for some n . Assume this is the least
 677 such n . Then by induction, going backwards from n , we give a sequence of stacks
 678 S_m and traces s_m and t_m for all $m < n$ such that $S_0 = S$, $s_0 = \varepsilon$, $s_m t_m = t$, and
 679 $t_m, S_m, \mu(M^i) \Downarrow T, j$, and $(S_m, i)_{s_m} \in \mathcal{T}_m$ for $m \neq 0$.

* For the case $m = n - 1$, since $(T, j)_t \in \mathcal{T}_n$, by definition it is either in $\mathcal{T}_{n-1} \setminus i$, which is ruled out by assumption, or it is given by an evaluation $R, \mu M \Downarrow \mathcal{T}_r$ with $R = S$ if $n = 1$, and otherwise $(R, i)_s \in \mathcal{T}_{n-1}$ and $t = sr$, with the evaluation below left. Let $S_{n-1} = R$, $s_{n-1} = s$, and $t_{n-1} = r$. Since $i \neq j$, this gives the derivation below right.

$$r, R, \mu M \Downarrow T, j \quad \frac{t_{n-1}, S_{n-1}, \mu M \Downarrow T, j}{t_{n-1}, S_{n-1}, \mu(M^i) \Downarrow T, j} \text{exit}$$

* For the cases $0 < m < n$, by induction let $(S_m, i)_{s_m} \in \mathcal{T}_m$ with $t = s_m t_m$ and the evaluation below left. By the definition of $\mathcal{T}_m$, $(S_m, i)_{s_m}$ is given by an evaluation $R, \mu M \Downarrow \mathcal{T}_r$ with $R = S$ if $m = 1$ and otherwise $(R, i)_s \in \mathcal{T}_{m-1}$ with $s_m = sr$, giving the evaluation below right.

$$t_m, S_m, \mu(M^i) \Downarrow T, j \quad r, R, \mu M \Downarrow S_m, i$$

To show the statement for $m - 1$, let $S_{m-1} = R$, $s_{m-1} = s$, and $t_m = r t_{m-1}$, and construct the derivation below.

$$\frac{s_{m-1}, S_{m-1}, \mu M \Downarrow S_m, i \quad t_m, S_m, \mu(M^i) \Downarrow T, j}{t_{m-1}, S_{m-1}, \mu(M^i) \Downarrow T, j} \text{loop}$$

It follows that for every $(T, j)_t \in \mathcal{T}$ there is an evaluation $t, S, \mu(M^i) \Downarrow T_t, j_t$, and hence $S, \mu(M^i) \Downarrow \mathcal{T}$.

■ For all $n \in \mathbb{N}^+$ we have $\mathcal{T}_n \in \text{RUN}(q_n \cdot \delta + \bar{\sigma} \cdot p^n i)$ and hence $\mathcal{T}_n \setminus i \in \text{RUN}(q_n \cdot \delta)$. Since each T in $\text{stk}(\mathcal{T}|j)$ is in some $\text{stk}(\mathcal{T}|j)$ we have $T \in \text{RUN}(\bar{\tau})$ where $\bar{\tau}$ is the stack for j in δ . Next, for $j \neq i$ we have $\text{pr}(\mathcal{T}_n|j) \geq q_n p_j$ where $q_n = 1 + p + p^2 + \dots + p^{n-1}$ and p_j is the probability for j in δ . This gives the following.

$$\text{pr}(\mathcal{T}|j) \geq (1 + p + p^2 + p^3 \dots) \times p_j = \frac{1}{1-p} p_j$$

It follows that $\mathcal{T} \in \text{RUN}(\frac{1}{1-p} \cdot \delta)$ and hence $\mu(M^i) \in \text{RUN}(\bar{\sigma} \Rightarrow \frac{1}{1-p} \cdot \delta)$.

◀